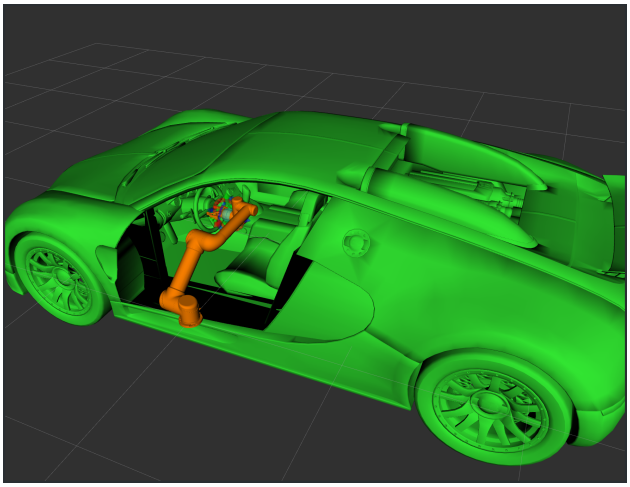

Figure 1: The UR5 enterring a car frame.

Assignment 3 (EN.601.663)

Simon Leonard

Due: March 26th 2026

1 Introduction

In this assignment you will implement a Probabilistic Road Map (PRM) path planner to plan collision-free motions of a UR5 robot. Your planner will use the MoveIt! libraries to modify an existing package and edit code to finalize the implementation. Moveit! is a large library with lots of classes and methods that are available to determine collisions and compute distances between objects.

Although it is not necessary, you are welcome to use external libraries such as Boost, FLANN and C++ standard template libraries (STL). You must ensure, however, that these libraries are installed by default with ROS desktop and your ROS package must be configured adequately. In other words,your TAs will not install additional software or configure their systems to use your package. So use external libraries carefully.

This is the most ambitious assignment from a programming point of view. In the past students encountered problems with Rviz and MoveIt!. We recommend that you at least try Section 3 early to make sure that all ROS packages are installed and you can use MoveIt!.

Section 3 is a small tutorial on how to use MoveIt! and the rviz plugin. No mark is associated with this section but try to get it out of the way (it only takes a few minutes) and it will expose any problem with your system configuration (i.e. missing packages, OpenGL) early on.

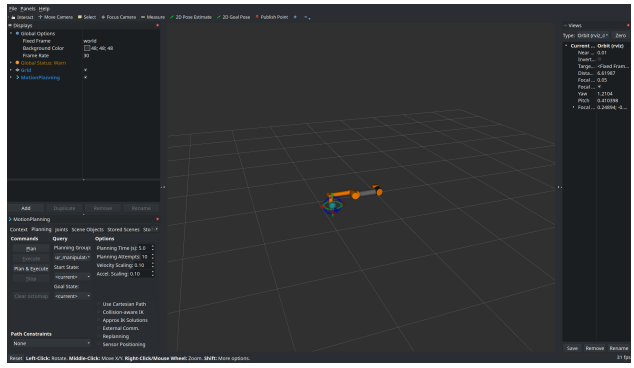

Figure 2: The MoveIt! display in RViz.

2 Delivirables

Your entire package `assignment3.tgz` that contains your source code and package configuration. Please ensure that your submission has all your latest files, that it compiles and runs cleanly. A penalty of 1 mark will be deducted if your package does not build.

3 MoveIt!

MoveIt! is a software for robot manipulation. It provides several tools for kinematics, sensing and path planning through other software (KDL, OMPL, FCL, SBPL, etc.) and is integrated with ROS and Rviz.

To install all the packages for this assignment you will need to enter the following command (you do not need this on Wyman workstations)

```
sleonard@percival:~$ sudo apt-get -y upgrade
sleonard@percival:~$ sudo apt-get -y install ros-humble-moveit* ros-humble-ur-*
```

Install the assignment in the usual workspace by using

```
sleonard@merliver:~$ wget www.cs.jhu.edu/~sleonard/sbr-a3.sh
sleonard@merliver:~$ chmod 755 sbr-a3.sh
sleonard@merliver:~$ ./sbr-a3.sh
```

Then try MoveIt! by entering the following command

```
sleonard@merliver:~$ ros2 launch ur_moveit_config ur_moveit.launch.py
    ur_type:=ur5e
```

You should see the environment illustrated in Fig. 2. On the bottom left you will find the MoveIt! panel with several tabs.

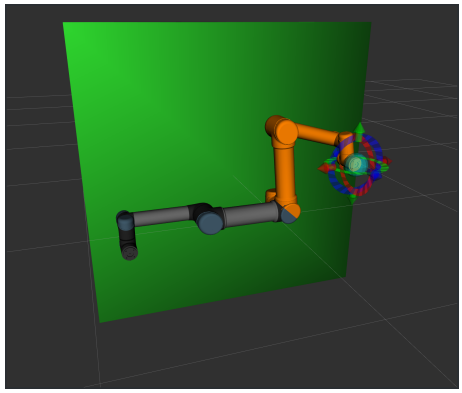

Figure 3: Scene with object and goal position.

3.1 Add Objects

First, add objects to your scene. In the "Scene Objects" tab, select "Mesh from file" in the drop down menu near the bottom and click on "+". A dialog window will allow you to search for 3D objects that you can import in your scene. You can use your own objects or you can use the file `panel.dae` provided in the directory `assignment3/meshes`. The object will appear green in the 3D view panel with an interactive marker. Interactive markers have arrows and wheels that allow to translate and rotate the object by clicking and dragging them. Move the object near the arm as illustrated in Fig. 3. Once you are done, click the `Publish Scene` button, then select the `Context` tab, select a planning algorithm from the `OMPL` drop down menu.

Important Note

To send your scene to the planner you need to click on `Publish Scene` each time you modify your scene. Otherwise, the planner will use the last published scene.

3.2 Set a Goal Configuration

Now you need to define a goal configuration of the robot. Notice that there is another interactive marker at the wrist of the robot. The marker enables you to move the end-effector of a robot to a goal configuration by using the arrows and wheels of the marker. Likewise, you can select a `random valid` goal state by clicking on the `Update` button on the `Planning` tab. This will generate a valid (collision-free) random goal position of the robot and display the robot in orange.

3.3 Plan the Motion

To plan the motion, click on the `Planning` tab and click on the `Plan` button. If the planner is successful, the robot will move from the start position to the goal position by avoiding the objects. If the planner fails to find a path, a `Failed` message will be displayed below the `Plan and Execute`

button. In this case you can try to plan again by clicking the `Plan` button or you can change the goal state or change the planning algorithm.

When you are done, type CTRL-C in the terminal where you launched `demo.launch`.

4 Sampling-Based Planner

You are now ready to implement a sampling based planner. You can compile and run the empty planner with the commands

```
sleonard@merliver ~/sbr_workspace$ colcon build --packages-select assignment3
sleonard@merliver ~/sbr_workspace$ ros2 launch assignment3 ur5e_robotiq.launch.py
```

In the `Context` tab, the first difference you will notice is that all the `OMPL` planners have been replaced by the `ASBR` planner. In the current implementation, the planner does not do anything and will return a failure.

4.1 Implementation

In this assignment, your only task is to implement a PRM planner. The implementation of the planning algorithm is in the file `src/assignment3_context.cpp`. Each time you click on the `Plan` button, the method `solve` is called. The `solve` method calls several other methods that you will implement. More specifically, *you must implement the following methods of a PRM planner*. Refer to the comments in the file `src/assignment3_context.h` to find the arguments to these methods. You can (and probably should) change the arguments and return type of these methods if you want. Additionally, you can add members (variables or methods) to the class if you need them.

1. `make_vertices`: Create a set of collision-free vertices V. Each joint is limited to $\pm\pi$. (1pt)
2. `make_edges`: Create a set of collision-free edges E connecting the vertices in V (2pt)
3. `search_accessibility`: Find a collision-free edge between the start position and a vertex v_s in V (accessibility) (1pt)
4. `search_departability`: Find a collision-free edge between the final position and a vertex v_f in V (departability) (1pt)
5. `search_path`: Find a path between v_s and v_f (connectivity) (3pt)
6. Expert challenge: Tune your planner to solve a difficult problem (1 pt).

A few utility methods are provided. Notably the method `state_collides` tests if a robot configuration collides with the scene and `interpolate` interpolates between two endpoints (i.e. between $\mathbf{q}_A$ and $\mathbf{q}_B$).

You are free to create your own data structures and define your own helper functions and methods, *but you must implement the above six methods* (you can change the signature of these methods

to fit your needs). You are also free to used third party libraries as long as they are part of a "stock" ROS desktop installation and your package is properly configured (graders will not install anything on their systems to test your code). You can also look at the MoveIt! API at https://moveit.picknik.ai/humble/index.html. In particular, the classes RobotModel and RobotState are highly relevant to this assignment (but not essential).

4.2 Expert Challenge

One point will be awarded if your planner has a success rate of at least %50 (4 attempts) with an "acceptable" planning time for a "difficult" planning problem. Planning time varies between systems but should be less than 30 seconds on a reasonable computer (i.e. Wyman 170). Note that we won't enable any optimization switches (release mode) to make your code run faster (we will use plain "colcon build"). An example of a difficult task is to enter the robot arm through the door of the car provided with this assignment and position the end-effector near the steering wheel (the finger tips about 10cm behind the wheel). To load the car scene, select the `Scene Objects` tab, then click on `Import`. In the dialog window, select the file `assignment3/meshes/bugatti.scene`.

You may have to publish the model a second time for this to work (students have reported that collisions are not detected without doing this). To publish a second time, you can add and remove a box to enable the "Publish" button or you can move the car a little.

4.3 Submission

You need to create and submit your assignment3 package by using the following command.

```
sleonard@merliver:~$ tar cvfz ~/assignment3.tgz -C ~/sbr_workspace/src/sbr
    assignment3
```

This will create the file assignment3.tgz. Ensure that your code is correct before submitting this file to Canvas.

A submitted package that does not build with "colcon build" will be deducted 1 mark. A submitted package that is not created properly (i.e. not using the command above) will be deducted 1 mark. A penalty of 10% (1 mark) per day will be applied to a late submission.

Reminder: By submitting your assignment you acknowledge that you have read the JHU Academic Misconduct Policy and that you are the author of 100% of the code submitted. Use of any code generation/completion software is not allowed. Furthermore, distributing or sharing your assignments (whereas online, with classmates or students that will take this course in future semesters) is not authorized and will be severely sanctioned.